DEEP LEARNING PROJECT REPORT

SMS Spam Classification with Character-Level Robustness

A controlled comparison of sparse text features and a compact character-level CNN

STUDENT	COURSE	DATE
Tejaswi Chava	COMP-671 Deep Learning	October 2026

Abstract

SMS spam filtering is a binary classification problem with asymmetric errors: missed spam is undesirable, while false alarms can hide legitimate messages. This project compares word-and-character TF-IDF logistic regression with a compact character-level TextCNN on the UCI SMS Spam Collection. It also evaluates whether mild character augmentation improves robustness on a deterministic text-corruption stress test. Exact normalized duplicates are kept in one partition; thresholds are selected on validation data under a ham false-positive-rate constraint, and final metrics are measured on held-out test data. In the regenerated run, TF-IDF achieved 0.991 spam recall on clean and perturbed test inputs, with ham false-positive rates of 0.047 and 0.050. The TextCNN variants had lower average precision and higher perturbed false-positive rates. Augmentation did not improve recall over the plain CNN; it modestly reduced the CNN's perturbed false-positive rate. These results show that a simple, well-chosen baseline can outperform a deep model on a small text dataset. They also demonstrate why validation-based operating-point selection, controlled comparisons, failure analysis, and limits on claims matter in applied deep learning.

Keywords: SMS filtering; text classification; character CNN; TF-IDF; robustness; evaluation

1. Problem Statement and Motivation

The task is message-level spam screening: given one SMS text, predict whether it is spam or legitimate ham. The input is a message string; the output is a spam probability and a binary decision made at a selected threshold. The intended role is decision support for human review, not automatic deletion or blocking. This framing makes the different costs of false negatives and false positives explicit.

Spam screening matters because unwanted messages consume attention and can facilitate fraud. Yet flagging a legitimate message can hide time-sensitive communication. Accuracy alone can conceal this tradeoff, especially when spam is the minority class. The primary operating objective is therefore spam recall at a validation-selected threshold constrained to ham false-positive rate (FPR) no greater than 5%. Average precision (PR-AUC) is a secondary threshold-independent measure of ranking quality. The held-out test metrics show how the selected policy transfers to unseen messages.

The project asks whether a character-level TextCNN improves over a TF-IDF logistic-regression baseline; whether character augmentation changes performance on reproducibly perturbed inputs; and what failure modes and limits appear beyond aggregate scores. The controlled ablation changes only training augmentation while holding model architecture, split, training configuration, threshold protocol, and neural seeds constant.

The scope is feasible for a semester course project: the corpus is small enough for local training, each inference is a single-message prediction, and the experiments need no paid service or specialized hardware. The end-to-end workflow ingests data, preprocesses text, trains and validates models, evaluates on held-out data, writes aggregate artifacts, and supports inference. The goal is not state-of-the-art performance. A meaningful result is a defensible comparison showing whether added model complexity improves the chosen application metric and where it fails.

A five-percentage-point increase in perturbed spam recall over the plain TextCNN was set as the intended meaningful effect for the augmentation question. This was not achieved. The negative result is reported directly rather than converted into a positive claim.

2. Data and Evaluation Protocol

The UCI SMS Spam Collection contains 5,574 labeled English messages [1]. It has 4,827 ham and 747 spam examples. This historical corpus is useful for a reproducible benchmark but does not represent current mobile traffic, all languages, or a specific carrier population.

The current run uses a stratified split by normalized exact-text groups with seed 42. Exact duplicates after case-folding and trimming remain within one partition to reduce leakage. The actual split is 3,918 training messages (3,391 ham, 527 spam), 828 validation messages (717 ham, 111 spam), and 828 test messages (719 ham, 109 spam). The source archive downloads from UCI and raw data are excluded from Git.

Feature fitting and neural training use only the training partition. Validation supports early stopping and selects the threshold that maximizes spam recall while meeting the 5% ham-FPR constraint. That threshold is frozen before final test evaluation. The test set is not used to tune the operating point. The test FPR is reported even if it exceeds the validation target; the target is not a promise of future performance.

The clean test condition uses original held-out text. The perturbed condition applies a fixed deterministic character-edit transform, with per-message seeds. The augmentation condition applies mild character changes only to its training examples. These synthetic edits probe a narrow form of spelling variation, not every evasion strategy.

The group split prevents exact normalized duplicates crossing partitions, but no sender or time identifiers exist to support sender-held-out or temporal evaluation. Messages from the same campaign could therefore remain related across partitions. A time-based external evaluation would better measure deployment generalization but cannot be constructed from these fields.

3. Method and Model Design

The baseline combines word and character TF-IDF n-grams. Word unigrams and bigrams capture lexical items and short phrases. Character n-grams of widths two through five can preserve partial words, punctuation, and spelling fragments. The sparse features are concatenated and passed to class-balanced logistic regression. This is a reasonable baseline for small text datasets: it is efficient and often captures recurring label-associated fragments with few learned parameters.

The neural model is a character-level TextCNN. Text is lowercased, encoded with a fixed printable-character alphabet, and truncated or padded to 256 characters. A trainable 16-dimensional embedding feeds parallel 1D convolutions of widths 3 and 5, each with 24 filters. ReLU activations and masked global max pooling summarize local features. Concatenated features pass through dropout of 0.30 and a binary output layer. The architecture follows the general local-pattern approach of CNN sentence classification [3], adapted from words to characters.

Training uses Adam, learning rate 0.001, batch size 256, weighted binary cross-entropy, and at most 10 epochs. Validation average precision chooses the retained checkpoint, with early stopping patience 2. The neural seeds are 17, 42, and 73. The plain and augmented TextCNN conditions share these settings. The ablation tests whether mild character changes in training help the same architecture handle the fixed perturbed test copy.

This method aligns with course topics in representation, supervised training, validation, CNN architectures, evaluation, and applied model improvement [2]. The project also tests an important practitioner assumption: a modern neural architecture is not automatically better than a carefully chosen simpler baseline.

4. Experimental Setup and Metrics

The fixed train/validation/test protocol separates model fitting, operating-point choice, and final evaluation. The baseline is one deterministic fit; each neural condition has three training seeds. CNN results are summarized as mean ± standard deviation across seeds. The baseline is a point estimate, not a mean over repeated runs. These summaries are descriptive; three seeds do not support strong significance claims.

The primary operational measures are spam recall and ham FPR at the validation-selected threshold. Recall is the share of spam messages caught; FPR is the share of legitimate messages flagged. The validation FPR constraint is at most 5%, but the test FPR can be higher. Average precision summarizes the precision-recall ranking over thresholds and is useful for an imbalanced positive class. Spam precision, macro-F1, and confusion counts are also computed.

The clean and perturbed conditions use identical test labels, enabling a paired comparison for this particular deterministic transform. The two TextCNN conditions use matched seeds and training settings. The predeclared practical goal was at least a five-point perturbed recall gain from augmentation. Accuracy is not emphasized because the majority ham class could make an unhelpful spam detector appear strong.

The implementation records configuration, partition sizes, per-run metrics, aggregate metrics, a plot, checkpoints, and a local error file. The README describes dataset retrieval, installation, commands, and project organization. Raw records, checkpoints, and error text stay out of version control. Re-running may vary slightly across hardware and dependency versions.

5. Results

Table 1 compares test performance at validation-selected thresholds. Neural results are mean ± standard deviation over three seeds; baseline is one fit.

Model	Clean PR-AUC	Clean recall	Clean ham FPR	Perturbed PR-AUC	Perturbed recall	Perturbed ham FPR
TF-IDF + logistic regression	0.9915	0.9908	0.0473	0.9895	0.9908	0.0501
TextCNN, plain	0.9705 ± .0057	0.9694 ± .0140	0.0566 ± .0045	0.9606 ± .0013	0.9694 ± .0140	0.1085 ± .0252
TextCNN, augmented	0.9708 ± .0061	0.9633 ± .0183	0.0515 ± .0037	0.9612 ± .0009	0.9602 ± .0191	0.0960 ± .0301

Table 1. Held-out test results. Ham FPR is the fraction of legitimate messages predicted as spam.

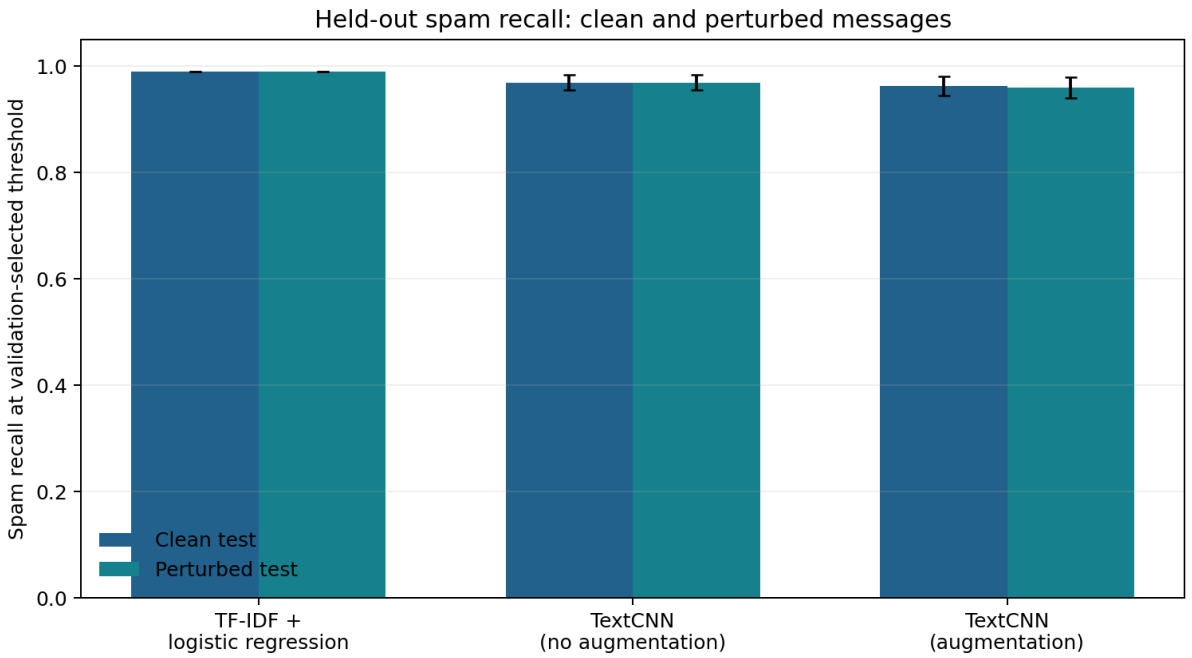


Figure 1. Spam recall on clean and fixed character-perturbed test messages. Bars show the mean; CNN error bars show across-seed standard deviation.

TF-IDF has the highest average precision and recall in both conditions. The test ham FPR is 0.047 clean and 0.050 perturbed, with 34 and 36 false positives respectively. The CNN variants have lower mean recall and PR-AUC. The perturbation mainly raises CNN false alarms: mean perturbed FPR is 0.1085 for plain training and 0.0960 for augmentation.

6. Failure Analysis and Interpretation

The clean baseline confusion counts were 34 ham false positives and one spam false negative. On perturbed text the baseline had 36 false positives and one false negative. Thus test spam recall was 0.991 in both conditions, while test ham FPR moved from 0.0473 to 0.0501. The perturbation leaves recall high but moves the false-alarm rate just above the validation constraint. These are still limited counts from a single split, not a production guarantee.

A reproduced augmented-TextCNN miss provides a concrete example: for the message ‘Congratulations, claim your prize now’, the checkpoint for seed 42 assigned spam probability 0.4763, below its selected 0.4839 threshold, and predicted ham. This shows that even clear promotional phrasing can fall just below a learned decision boundary. It is one demonstration, not a representative estimate of error types.

The plain TextCNN's mean perturbed ham FPR increased from 0.0566 clean to 0.1085 perturbed; the augmented variant increased from 0.0515 to 0.0960. The model appears sensitive to the character edits in a way that increases false flags on legitimate examples. Training augmentation reduces that increase slightly, but the augmented perturbed FPR remains nearly twice the baseline point estimate.

A plausible explanation for the baseline's advantage is that recurring phrases and fragments in this small corpus are captured directly by n-grams. The CNN must learn local filters and a decision boundary from fewer than four thousand training records, including only 527 spam messages. Class weighting improves the training loss balance but does not guarantee calibrated probabilities or robustness under distribution shift. These are explanations consistent with the measurements, not causal proofs.

The raw error-analysis CSV contains original SMS text and is ignored by Git because messages may reveal personal details. Public reporting uses aggregate counts. An illustrative miss should only be included after it is reproduced from the current local checkpoint and reviewed for identifying information. More seeds, bootstrap intervals, a time-based split, and redacted qualitative coding would strengthen the analysis.

7. Ethics, Limitations, and Responsible Use

SMS is personal communication, even in a public benchmark. Message text may contain names, contact details, locations, or sensitive context. Raw samples and error rows should not be published without privacy review and redaction. The repository ignores raw data and local message-level error outputs; the report uses aggregate metrics rather than reproducing private messages.

False positives can hide a legitimate message from a family member, school, employer, service, or account provider. A validation constraint does not guarantee safety on a held-out set or future traffic. A deployed tool should preserve quarantined messages, make decisions reversible, allow users to correct or appeal labels, and monitor false alarms. This prototype should not be the sole decision-maker for consequential communication and should not automatically delete messages.

The corpus is historical and English-language. Legitimate dialect, shorthand, multilingual messages, accessibility-related language, or new vocabulary may differ from training examples. No demographic attributes are available, so the project cannot make subgroup fairness claims. A random split measures interpolation within this corpus rather than fairness or future performance.

Synthetic character edits provide only a narrow robustness check and may not reflect natural variation or adversarial behavior. The 256-character limit truncates longer messages. No sender grouping or temporal holdout is possible. Probability calibration under changing spam prevalence is not evaluated. The baseline is more efficient and outperforms the deep models here, so there is no measured application reason to prefer the CNN beyond studying the course architecture.

Responsible use would require human review, monitoring, correction pathways, representative new data, and privacy-protective evaluation. This is an educational benchmark, not a production recommendation.

8. Conclusion and Reproducibility

This project implemented a full SMS classification workflow and compared a sparse TF-IDF baseline, two TextCNN training conditions, and a deterministic text-perturbation stress test. It uses duplicate-aware stratified splits, validation-only threshold selection, held-out testing, and three seeds for neural models.

The main finding is that TF-IDF logistic regression has the strongest ranking and recall in this regenerated run. It achieves 0.991 test spam recall and PR-AUC of 0.992 on clean data; perturbed recall remains 0.991 and PR-AUC is 0.990. The test ham FPR is 0.047 clean and 0.050 perturbed. The TextCNNs have lower average precision and materially higher perturbed false-positive rates. Augmentation does not reach its predefined recall improvement target, although it modestly lowers the CNN's perturbed FPR.

This conclusion is bounded to one historical corpus, split, model size, and synthetic edit. It does not establish universal superiority of TF-IDF, realistic adversarial robustness, or readiness for live filtering. Future work should evaluate new, time-separated data, more seeds, uncertainty, threshold calibration, and privacy reviewed errors.

To reproduce, create a Python 3.9+ virtual environment, install requirements, install the package in editable mode, and run `python -m sms_spam.cli run --output-dir outputs`. The command downloads the UCI dataset, trains and evaluates all three conditions, and writes results and local checkpoints. Use the documented `predict` command with a saved TextCNN checkpoint for single-message inference. Aggregate metrics and split metadata are tracked in the reports directory; raw messages and checkpoints remain local.

The work follows the applied-learning cycle in the course: formulate an application problem, justify data representation and models, train with validation feedback, compare to a baseline, analyze failures, and state ethical and generalization limits [2].

References

[1] Almeida, T. A., Hidalgo, J. M. G., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. In *Proceedings of the 11th ACM Symposium on Document Engineering* (pp. 23–32). https://doi.org/10.1145/2034691.2034742. Dataset record: UCI Machine Learning Repository, SMS Spam Collection, https://doi.org/10.24432/C5CC84.

[2] Prince, S. J. D. (2023). *Understanding Deep Learning*. The MIT Press. https://udlbook.github.io/udlbook/.

[3] Kim, Y. (2014). Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing* (pp. 1746–1751). Association for Computational Linguistics. https://doi.org/10.3115/v1/D14-1181.

[4] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html.

Report note

This PDF was regenerated from the measured experiment outputs in the accompanying repository. UCI attribution is retained. Message-level raw examples are omitted for privacy.